

EDS Lab Assignments

Practical 1

Name: Keshav Devaji Bahekar

PRN: 202501040031

1.1.1 Calculate Momentum

Problem Statement:

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum is calculated using the formula: $p = m \times v$

where:

m is the mass of the object (in kilograms).

v is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Code:

```
mass = float(input())
velocity = float(input())
momentum = mass * velocity
print(f"{momentum:.2f} kgm/s")
```

Output:

The screenshot displays the CODETANTRA IDE interface for the '1.1.1. Calculate Momentum' assignment. The left sidebar contains the problem statement, which includes the formula $p = m \times v$ and defines m as mass and v as velocity. It also specifies the input and output formats. The main editor shows the Python code for calculating momentum. The right sidebar displays the test results, indicating that 2 out of 2 shown test cases passed. The test cases show expected and actual outputs for mass and velocity inputs, resulting in the correct momentum values.

Test Case	Expected Output	Actual Output
Test case 1	5.0 10.0 50.00kgm/s	5.0 10.0 50.00kgm/s
Test case 2	2.3 4.8 11.04kgm/s	2.3 4.8 11.04kgm/s

1.1.2 Conditional Calculation based on the Number of Digits

Problem Statement:

Write a Python program that accepts an integer as input. Depending on the number of digits in.

Constraints: $1 \leq n \leq 999$

Input Format:

- The input consists of a single integer.

Output Format:

- If is a single-digit number, print its square.
- If is a two-digit number, print its square root (rounded to two decimal places).
- If is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid"

Code:

```
import math
n=int(input())
sq = n*n
sq_root = math.sqrt(n)
cb_root = math.cbrt(n)
if n>1 and n<=9:
    print(f"{sq}")
elif n>=10 and n<=99:
    print(f"{sq_root:.2f}")
elif n>=100 and n<=999:
    print(f"{cb_root:.2f}")
else:
    print("Invalid")
```

Output:

The screenshot displays the DE TANTRA online IDE interface. On the left, the problem statement and constraints are visible. The main editor shows the Python code for calculating the square, square root, or cube root based on the number of digits. The right sidebar shows the test results, indicating that 4 out of 4 shown test cases passed and 3 out of 3 hidden test cases passed. The test cases table shows the expected and actual outputs for each case.

Test Case	Expected Output	Actual Output
Test case 1	81	81
Test case 2	166	166

1.1.3 Days Between Two Dates

Problem Statement:

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format YYYY-MM-DD.
- The second line contains the second date in the format YYYY-MM-DD.

Output Format:

- An integer representing the number of days between the given dates.

Code:

```
from datetime import date
from datetime import datetime
date1 = input().strip()
date2 = input().strip()
d1 = datetime.strptime(date1, "%Y-%m-%d")
d2 = datetime.strptime(date2, "%Y-%m-%d")
difference = d2-d1
print(difference.days)
```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, the problem statement is shown, including the input/output formats and a note. The main editor area contains the Python code for calculating the number of days between two dates. The right sidebar shows the test results, indicating that 2 out of 2 shown test cases passed. The test cases are as follows:

Test Case	Expected Output	Actual Output
Test case 1	2014-07-02 2014-11-02 123	2014-07-02 2014-11-02 123
Test case 2	2014-07-02 2014-07-11	2014-07-02 2014-07-11

1.1.4 Reverse a Number

Problem Statement:

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

Code:

```
n=int(input())
reverse=int(str(n)[::-1])
print(reverse)
```

Output:

The screenshot displays a coding platform interface for the problem '1.1.4. Reverse a Number'. The problem statement asks to write a Python program to reverse the digits of a number and print the reversed number. The input format is a single integer, and the output format is a single integer representing the reversed number. The code provided is:

```
n=int(input())
reverse=int(str(n)[::-1])
print(reverse)
```

The interface shows the code is executed successfully, with the following performance metrics:

Metric	Value
Average time	0.011 s
Maximum time	0.017 s
Test case 1 (Expected: 5367, Actual: 5367)	47 ms
Test case 2 (Expected: 8132, Actual: 8132)	4 ms

The interface also shows that 2 out of 2 shown test case(s) passed and 3 out of 3 hidden test case(s) passed. The bottom of the interface includes navigation buttons: < Prev, Reset, Submit, and Next >.

1.1.5 Multiplication Table

Problem Statement:

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:
<number> x <multiplier> = <result>

Code:

```
n = int(input())
```

```
for i in range(1,11):
```

```
    print(n, "x", i, "=", n*i)
```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, the problem statement for '1.1.5. Multiplication Table' is shown, including the input and output formats. The main editor area contains the following Python code:

```
1 n = int(input())
2 for i in range(1,11):
3     print(n, "x", i, "=", n*i)
```

Below the code, the execution results are shown. The average time is 0.011 s and the maximum time is 0.015 s. The output indicates that 2 out of 2 shown test case(s) passed and 2 out of 2 hidden test case(s) passed. The first test case shows the expected output for n=8:

```
8 x 1 = 8
8 x 2 = 16
8 x 3 = 24
8 x 4 = 32
8 x 5 = 40
8 x 6 = 48
8 x 7 = 56
8 x 8 = 64
8 x 9 = 72
8 x 10 = 80
```

The actual output matches the expected output. The second test case also shows a match between expected and actual output.

1.2.1 Pass or Fail

Problem Statement:.

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer, the number of courses.
- The second input will be integers representing the marks of the student in each of the courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Code:

```
def calculate(marks, total_courses):
    if any(mark<40 for mark in marks):
        return "Fail"
    total_marks = sum(marks)
    aggregate_percentage = ((total_marks)/(total_courses*100))*100
    if aggregate_percentage > 75:
        grade = "Distinction"
    elif aggregate_percentage >= 60:
        grade = "First Division"
    elif aggregate_percentage >= 50:
        grade = "Second Division"
    elif aggregate_percentage >= 40:
        grade = "Third Division"
    return (aggregate_percentage, grade)
num_courses = int(input())
marks = list(map(int,input().split()))
result = calculate(marks, num_courses)
if result == 'Fail':
    print("Fail")
else:
```

```

aggregate_percentage, grade = result
print("Aggregate Percentage:",f"{aggregate_percentage:.2f}")
print(f"Grade: {grade}")

```

Output:

[Home](#)
[Learn Anywhere](#)

202501040043@mitaoe.ac.in
Support
Logout

1.2.1. Pass or Fail

Write a Python program that accepts the number of courses and the marks of a student in those courses. The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer n , the number of courses.
- The second input will be n integers representing the marks of the student in each of the n courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Note:

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

Sample Test Cases

passorFa...

```

1 def calculate(marks, total_courses):
2     if any(mark < 40 for mark in marks):
3         return "Fail"
4     total_marks = sum(marks)
5     aggregate_percentage = ((total_marks)/(total_courses*100))*100
6
7     if aggregate_percentage > 75:
8         grade = "Distinction"
9     elif aggregate_percentage >= 60:
10        grade = "First Division"
11    elif aggregate_percentage >= 50:
12        grade = "Second Division"
13    elif aggregate_percentage >= 40:
14        grade = "Third Division"
15    return (aggregate_percentage, grade)

```

Average time
0.029 s
28.80 ms

Maximum time
0.036 s
36.00 ms

5 out of 5 shown test case(s) passed
5 out of 5 hidden test case(s) passed

Test case 1

Expected output

Actual output

4

4

55.65 78.89

55.65 78.89

Aggregate Percentage: 71.75

Aggregate Percentage: 71.75

Grade: First Division

Grade: First Division

Terminal
Test cases

< Prev
Reset
Submit
Next >

1.2.2 Fibonacci Series

Problem Statement:

Write a Python program that uses recursion to print the first terms of the Fibonacci series.

Input Format:

- A single integer representing the number of terms to generate.

Output Format:

- A single line containing the first terms of the Fibonacci sequence, separated by spaces.

Code:

```
def fibonacci(n, a=0, b=1):  
    if n==0:  
        return a  
    else:  
        return fibonacci(n-1,b,a+b)
```

```
n=int(input())  
for i in range(n):  
    print(fibonacci(i),end=" ")
```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, the problem statement for '1.2.2. Fibonacci Series' is shown, including the input and output formats. The main editor area contains the Python code for the Fibonacci series using recursion. The code is as follows:

```
def fibonacci(n, a=0, b=1):  
    if n==0:  
        return a  
    else:  
        return fibonacci(n-1,b,a+b)  
  
n=int(input())  
for i in range(n):  
    print(fibonacci(i),end=" ")
```

Below the code editor, the test results are displayed. It shows that 2 out of 2 shown test cases passed and 2 out of 2 hidden test cases passed. The test cases are as follows:

Test Case	Expected Output	Actual Output
Test case 1	0 1 1 2 3	0 1 1 2 3
Test case 2	0 1 1 2 3 5 8 13 21 34 55 89 144 233 377	0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

The interface also includes a terminal, test cases section, and navigation buttons at the bottom.

1.2.3 Pattern-1

Problem Statement:

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Code:

```
rows = int(input())
for i in range(1, rows+1):
    for j in range(i):
        print("*",end = "")
    print()
```

Output:

The screenshot displays a coding platform interface for a problem titled "1.2.3. Pattern - 1". The problem statement asks for a Python program to print a right-angled triangle pattern of asterisks. The input format specifies an integer for the number of rows, and the output format specifies an increasing number of asterisks per row. The code editor shows the following Python code:

```
rows = int(input())
for i in range(1, rows+1):
    for j in range(i):
        print("*",end = "")
    print()
```

The interface shows the code has been executed successfully, with a message indicating "2 out of 2 shown test case(s) passed" and "4 out of 4 hidden test case(s) passed". The average time taken was 0.009 s, and the maximum time was 0.014 s. The output for two test cases is shown, both displaying the expected right-angled triangle pattern of asterisks.

1.2.4 Pattern-2

Problem Statement:

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Code:

```
n=int(input())
for i in range(1, n+1):
    for j in range(1, i+1):
        print(j,end=" ")
    print()
```

Output:

The screenshot shows the CodeTANTRA IDE interface. On the left, the problem statement and input/output formats are displayed. The main editor shows the Python code for printing the pattern. The terminal on the right shows the execution results, including the input '5' and the resulting pattern of numbers.

CodeTANTRA Home Learn Anywhere

1.2.4. Pattern - 2

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

- Refer to the displayed test cases for the sample pattern.

Sample Test Cases

numberP...

```
1 n=int(input())
2 for i in range(1, n+1):
3     for j in range(1, i+1):
4         print(j,end=" ")
5     print()
```

Average time: 0.015 s, Maximum time: 0.021 s, 15.00 ms, 21.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 (1/1) [Debug] [Run]

Expected output	Actual output
5	5
1	1
1 2	1 2
1 2 3	1 2 3
1 2 3 4	1 2 3 4
1 2 3 4 5	1 2 3 4 5

Test case 2 (1/1) [Debug] [Run]

Expected output	Actual output
8	8
1	1
1 2	1 2
1 2 3	1 2 3
1 2 3 4	1 2 3 4
1 2 3 4 5	1 2 3 4 5
1 2 3 4 5 6	1 2 3 4 5 6
1 2 3 4 5 6 7	1 2 3 4 5 6 7
1 2 3 4 5 6 7 8	1 2 3 4 5 6 7 8

Terminal Test cases

Prev Next